Informe del Proyecto Integrador - Cine Paradiso

Universidad de Buenos Aires - Facultad de Ingeniería (FIUBA)
Materia: Introducción al Desarrollo de Software - Cátedra Lanzillotta
Período: 2.º cuatrimestre de 2026
Grupo: 15
Proyecto: Cine Paradiso - Sistema web de venta de entradas de cine y gestión de salas
Versión de entrega: v1.0

Integrantes

Integrante | Rol en el equipo

Lucas Cirillo Berardi | Desarrollo full-stack y coordinación
Santiago Fernandez Bain | Desarrollo full-stack
Juan Pablo Bruzone | Desarrollo full-stack
Francisco Martinez | Desarrollo full-stack
Víctor Daniel Morinigo | Desarrollo full-stack
Nicolás Valenzuela | Desarrollo full-stack
Joaquín Nicolás Zavala Eguez | Desarrollo full-stack
Martín Simajowich | Desarrollo full-stack

Resumen

Cine Paradiso es una plataforma web integral para la comercialización de entradas de cine, servicios adicionales de confitería y estacionamiento, y la administración operativa de múltiples complejos cinematográficos. El sistema resuelve de punta a punta el recorrido del espectador -desde la consulta de la cartelera filtrada por sede y fecha, la selección interactiva de butacas en tiempo real, el agregado de productos adicionales y el pago simulado, hasta la recepción por correo electrónico de la factura con el código QR de ingreso y el enlace de cancelación- junto con el panel de administración (backoffice) para importar títulos desde la API de The Movie Database (TMDB), programar funciones sin solapamientos, gestionar locales, salas y servicios, validar el ingreso en puerta mediante lectura de QR y analizar métricas operativas en cuatro tableros de control.

Desde el punto de vista arquitectónico, la solución se estructura en dos aplicaciones Flask independientes que se comunican exclusivamente mediante HTTP/JSON a través de la librería requests desde el servidor, prescindiendo por completo de peticiones desde el navegador (fetch, axios o XMLHttpRequest), de mapeadores objeto-relacionales (ORM) y de programación orientada a objetos. La persistencia se apoya en una base de datos relacional MySQL 8 con consultas SQL explícitas y parametrizadas, transacciones atómicas y bloqueos a nivel de fila para garantizar que ninguna butaca se venda dos veces y que ningún código QR pueda utilizarse en más de una oportunidad.

1. Introducción

La operación de una cadena de cines con múltiples complejos y salas requiere coordinar en simultáneo la programación de funciones, la disponibilidad de asientos en tiempo real, la venta de productos complementarios y el control de acceso en sala. Sin un sistema centralizado que garantice consistencia transaccional, surgen problemas frecuentes: sobreventa de una misma butaca ante compras concurrentes, reservas abandonadas que bloquean asientos indefinidamente, cancelaciones tardías sobre funciones próximas a iniciar y reutilización fraudulenta de entradas en el ingreso.

El objetivo de este trabajo integrador es diseñar, implementar, probar y documentar una aplicación web completa para Cine Paradiso que resuelva estas necesidades bajo las restricciones técnicas y metodológicas de la cátedra:

1. Separación estricta entre interfaz web y API REST: un frontend renderizado en el servidor con Jinja2 y hojas de estilo responsive (mobile-first), y un backend REST en JSON conectado a MySQL.
2. Paradigma procedural y declarativo: código Python estructurado únicamente con funciones puras, módulos, diccionarios, listas y tuplas, sin clases ni herencia, tanto en la aplicación como en la batería de pruebas automatizadas.
3. Integridad de datos en la base relacional: uso de claves foráneas compuestas, restricciones UNIQUE e índices en MySQL, almacenando todas las marcas temporales en un único campo DATETIME.
4. Trabajo colaborativo trazable: organización ágil en sprints con tablero en Jira (SCRUM), documentación de alcance e hipótesis en Confluence, diseño previo de pantallas en Figma y flujo Git con ramas por tarea (SCRUM-XXX/descripcion) integradas mediante Pull Requests revisados por pares.

2. Solución Propuesta

2.1. Arquitectura general

El sistema se divide en dos aplicaciones Flask desacopladas, cada una creada mediante su propia fábrica create_app():

- Backend (src/backend/): expone la API REST bajo el prefijo /api. Sus rutas (src/backend/routes/) solo traducen peticiones y respuestas HTTP/JSON y delegan las validaciones y reglas de negocio en los módulos de servicios (src/backend/services/). El acceso a MySQL se concentra en src/backend/db.py mediante las funciones consultar, consultar_uno, ejecutar y el administrador de contexto transaccion().
- Frontend (src/frontend/): renderiza las vistas públicas del cliente (routes/publico.py) y el panel de administración (routes/backoffice.py). Toda comunicación hacia el backend pasa por una única función, api_client.llamar(metodo, ruta, json=, params=), que ejecuta la petición desde Python con requests, propaga el token de sesión y traduce cualquier error a la tupla (datos, error). El navegador solo envía formularios HTML estándar y sigue redirecciones (patrón POST / Redirect / GET).

2.2. Modelo de datos e invariantes de negocio

La base de datos cine_paradiso (src/backend/db/schema.sql) cuenta con once tablas relacionales (usuarios, locales, salas, butacas, peliculas, funciones, servicios, compras, entradas, compra_servicios y validaciones_ingreso). Las reglas críticas del dominio se respaldan tanto en el motor relacional como en transacciones del backend:

- Prohibición de doble venta de butacas: la tabla entradas define UNIQUE (funcion_id, butaca_id, activa) y una clave foránea compuesta (compra_id, funcion_id) hacia compras (id, funcion_id). Mientras la compra está vigente (pendiente o confirmada), activa vale 1; al vencer o cancelarse, pasa a NULL. Como MySQL permite múltiples valores NULL en un índice único, la butaca vuelve a quedar disponible sin borrar el registro histórico.
- Reserva temporal de 10 minutos (MINUTOS_RESERVA): al elegir butacas se crea una compra en estado pendiente con vencimiento vence_en. Antes de consultar disponibilidad o reservar, se marcan como vencida las reservas expiradas y se liberan sus entradas.
- Confirmación con QR firmado y factura por email: al confirmar el pago simulado, el backend firma el identificador de la compra con itsdangerous (SECRET_KEY y salt dedicado "qr"), genera la imagen PNG del código QR en memoria y envía por SMTP un correo multipart (texto plano y HTML con la imagen embebida por Content-ID y adjunta) que incluye el enlace

firmado de cancelación. Si el servidor SMTP falla, la compra queda confirmada igualmente e informa email_enviado: false.
- Cancelación con plazo límite (HORAS_LIMITE_CANCELACION): una compra confirmada y no utilizada puede cancelarse hasta 2 horas antes del inicio de la función, tanto desde el perfil del cliente como desde el enlace firmado del correo electrónico (sin exigir inicio de sesión, requiriendo confirmación explícita por POST).
- Validación de ingreso y auditoría: el endpoint POST /api/admin/ingresos/validar verifica en orden estricto la firma del token (QR_INVALIDO), el estado de la compra (CANCELADA), el uso previo (YA_UTILIZADA, informando fecha y hora del primer escaneo), la coincidencia de la función (OTRA_FUNCION) y la vigencia horaria (FUERA_DE_FECHA). El chequeo se ejecuta con bloqueo de fila (SELECT ... FOR UPDATE), registra cada intento -válido o rechazado- en validaciones_ingreso y marca utilizada_en = NOW().
- Programación de funciones sin solapamientos: al crear o modificar una función se verifica en la misma sala que no se superponga con otra función activa considerando la duración de la película más 15 minutos de limpieza entre proyecciones.
- Bajas lógicas: películas con funciones, salas con historial, funciones y servicios utilizan banderas booleanas (activa / activo) para preservar la integridad estadística de los tableros.

3. Tecnología Utilizada

Capa / Herramienta | Tecnología y versión | Uso en el proyecto

Lenguaje | Python 3.12+ | Desarrollo íntegro del backend, frontend y pruebas sin POO
Framework web | Flask 3.0.3 + Jinja2 | API REST en JSON (src/backend) y servidor de plantillas (src/frontend)
Base de datos | MySQL 8 / MariaDB + PyMySQL 1.1.1 | Persistencia relacional con SQL explícito parametrizado (%s) y DictCursor
Cliente HTTP | requests 2.32.3 | Comunicación servidor a servidor entre frontend y backend, y cliente de TMDB
Seguridad y criptografía | werkzeug.security (pbkdf2:sha256) + itsdangerous | Hash de contraseñas portable y firma criptográfica de tokens de sesión, QR y cancelación
Generación de QR y correo | qrcode 8.2 + pypng + smtplib / email.message | Generación de QR en PNG en Python puro y envío de facturas HTML por SMTP con STARTTLS/SSL
Interfaz de usuario | HTML5 semántico + CSS3 (Flexbox, Grid, variables CSS) | Diseño responsive mobile-first basado en el sistema de diseño de Figma
Pruebas automatizadas | pytest 8.3 + unittest.mock | Más de 900 pruebas unitarias y de integración sin requerir servicios externos
Gestión y diseño | Jira (SCRUM), Confluence, Figma, Git y GitHub | Planificación por sprints, documentación funcional, mockups y revisión de código

4. Conclusión

4.1. Decisiones de diseño

1. Dos aplicaciones Flask comunicadas únicamente por requests: separar el frontend y el backend obligó a definir desde el primer sprint un contrato explícito en docs/endpoints.md con un formato único de error ({"error": {"codigo", "mensaje"}}). Esto permitió desarrollar y probar las pantallas del frontend simulando las respuestas de la API mientras se construían las consultas SQL del backend en paralelo.
2. Programación procedural sin clases ni ORM: representar todas las entidades como diccionarios devueltos por DictCursor y estructurar los casos de uso como funciones por módulo (peliculas.py, funciones.py, compras.py, ingresos.py, dashboards.py) simplificó el flujo de datos y eliminó capas intermedias de serialización. Para evitar la repetición de código se

extrajeron validadores comunes (services/validaciones.py), macros de Jinja (templates/macros.html) y utilidades de prueba (tests/utilidades.py).
3. Tokens firmados independientes por propósito: tanto la sesión como el código QR de ingreso y el enlace de cancelación por email emplean firmas criptográficas con SECRET_KEY, pero con salts distintos ("qr" y "cancelar"). De este modo, el enlace de cancelación puede usarse desde el correo sin iniciar sesión y sin que ese token sirva para ingresar a la sala.
4. Resiliencia ante servicios externos: las integraciones con TMDB y con el servidor SMTP se aislaron en módulos propios con tiempos máximos de espera (timeouts). En particular, el envío de correo se ejecuta fuera de la transacción de confirmación de la compra para que una caída eventual del proveedor de email no revierta una entrada ya pagada.

4.2. Dificultades encontradas y resolución

- Concurrencia en la reserva de butacas y en el escaneo de QR: para evitar condiciones de carrera cuando dos clientes intentan reservar el mismo asiento o cuando un código QR se escanea dos veces simultáneamente, combinamos bloqueos pesimistas (SELECT ... FOR UPDATE) dentro de with transaccion() as cursor: con la restricción UNIQUE (funcion_id, butaca_id, activa) en la base de datos. Además, en la validación de ingresos se cuidó registrar el intento en validaciones_ingreso antes de cortar la petición con error_api, garantizando que el commit persista también los rechazos.
- Compatibilidad multiplataforma: durante las pruebas detectamos diferencias entre sistemas operativos (como la ausencia de hashlib.scrypt en instalaciones de Python sobre macOS o las rutas del entorno virtual en Windows frente a Linux). Se estandarizó el hash de contraseñas en pbkdf2:sha256, la generación de imágenes QR en Python puro con pypng y el script init.sh para detectar ambos entornos.
- Cobertura de pruebas sin dependencias externas: para que toda la suite de pytest corra en pocos segundos en cualquier máquina recién clonada, se construyó en src/tests/utilidades.py un simulador del módulo de base de datos que intercepta consultar, consultar_uno, ejecutar y transaccion() y falla de inmediato si alguna prueba intenta abrir una conexión real a MySQL.

4.3. Balance final

El proyecto permitió integrar en un producto funcional completo todos los ejes de la materia: modelado de datos relacional en MySQL, diseño de APIs REST seguras en Flask, construcción de interfaces web accesibles y responsive sin JavaScript de red, pruebas automatizadas exhaustivas y trabajo en equipo coordinado mediante Git, Pull Requests y metodología Scrum.